

Laboratory 08

TrustZone-M: Secure/Non-Secure Partition

Real-Time Operating Systems • M.Eng. ECE • KMUTNB

Platform	NXP FRDM-MCXM236 (ARM Cortex-M33 @ 150 MHz)
RTOS	FreeRTOS v10.6
Toolchain	ARM GNU Toolchain (<code>arm-none-eabi-gcc</code>) + Make
Estimated time	4–5 hours
CLO mapping	CLO 5 & CLO 6

1. Objectives

By completing this lab you will be able to:

1. **Configure** the SAU to partition flash and SRAM into Secure, NSC, and Non-Secure regions.
2. **Implement** a Secure API using the CMSE `__attribute__((cmse_nonsecure_entry))` annotation.
3. **Call** a Secure function from a Non-Secure FreeRTOS task via an NSC veneer.
4. **Trigger** a SecureFault by attempting a direct NS→Secure SRAM access.
5. **Measure** NSC call overhead vs. a direct function call using DWT.

2. Prerequisites

- ☐ Lab 07 complete (stack analysis and static allocation working).
- ☐ ARM GNU Toolchain with CMSE support (`arm-none-eabi-gcc` $\geq$ 10, compiled with `-mcmse`).
- ☐ MCUXpresso IDE or command-line build with separate Secure and Non-Secure projects.
- ☐ SEGGER J-Link or MCU-Link (J-Link firmware) for debugging SecureFault.

3. Background

3.1. TrustZone-M Execution Model

The Cortex-M33 has two security states: **Secure (S)** and **Non-Secure (NS)**. The processor is always in one state. Transitions happen only through specific instructions:

- **NS → S:** only through the SG (Secure Gateway) instruction at an address in a Non-Secure Callable (NSC) region.
- **S → NS:** via BXNS / BLXNS instructions.

3.2. Two-Project Build Structure

TrustZone-M requires two separate firmware images:

```
Secure project (linked to 0x0000_0000)
- Runs first after reset
- Configures SAU, TrustZone peripherals
- Implements secure API functions
- Releases Non-Secure image

Non-Secure project (linked to 0x0004_0000)
- FreeRTOS scheduler
- Application tasks
- Calls Secure API through NSC veneers
```

4. Project Structure

```

lab08_trustzone/
secure/                                <- Secure-world project
  Makefile
  linker_secure.ld                     <- Secure flash + SRAM regions
  src/
    main_secure.c                     <- SAU config, Secure init, NS release
    secure_api.c/.h                   <- NSC entry functions
    secure_api_veneer.h               <- auto-generated veneer declarations
    startup_MCXN236_s.s               <- Secure vector table
    cmse_implib.o                     <- output: NSC import library
nonsecure/                             <- Non-Secure FreeRTOS project
  Makefile
  linker_nonsecure.ld                 <- NS flash + SRAM regions
  src/
    main_ns.c                         <- FreeRTOS init, task creation
    ns_tasks.c/.h                     <- tasks that call Secure API
    FreeRTOSConfig.h
    uart.h / uart.c
  FreeRTOS-Kernel/
flash_both.sh                           <- flashes S image then NS image
src/
  uart.h                              <- shared UART declaration

```

5. Part A — SAU Configuration

5.1. Step 1 — Define the Memory Partition

Edit `secure/src/main_secure.c` to configure the Security Attribution Unit:

```

1 void SAU_Configure(void)
2 {
3     /* Disable SAU before configuration */
4     SAU->CTRL = 0;
5
6     /* Region 0: Non-Secure flash [0x0004_0000 .. 0x001F_FFFF] */
7     SAU->RNR = 0;
8     SAU->RBAR = 0x00040000U;
9     SAU->RLAR = (0x001FFFFFFU & SAU_RLAR_LADDR_Msk) |
10        SAU_RLAR_ENABLE_Msk;
11
12     /* Region 1: NSC region in flash [0x0003_FE00 .. 0x0003_FFFF]
13        (512 B) */
14     SAU->RNR = 1;
15     SAU->RBAR = 0x0003FE00U;
16     SAU->RLAR = (0x0003FFFFFFU & SAU_RLAR_LADDR_Msk)
17        | SAU_RLAR_NSC_Msk | SAU_RLAR_ENABLE_Msk;
18
19     /* Region 2: Non-Secure SRAM [0x2002_0000 .. 0x200F_FFFF] */
20     SAU->RNR = 2;
21     SAU->RBAR = 0x20020000U;
22     SAU->RLAR = (0x200FFFFFFU & SAU_RLAR_LADDR_Msk) |
23        SAU_RLAR_ENABLE_Msk;
24
25     /* Enable SAU */
26     SAU->CTRL = SAU_CTRL_ENABLE_Msk;
27     __DSB();
28     __ISB();
29 }

```

5.2. Step 2 — Verify with TZM_IsSecure

Add a diagnostic function that probes whether an address is Secure:

```

1  /* Secure-side only -- NS code cannot call this */
2  bool TZM_IsSecure(uint32_t addr)
3  {
4      for (int i = 0; i < 8; i++) {
5          SAU->RNR = i;
6          if (!(SAU->RLAR & SAU_RLAR_ENABLE_Msk)) continue;
7          uint32_t base = SAU->RBAR & SAU_RBAR_BADDR_Msk;
8          uint32_t limit = (SAU->RLAR & SAU_RLAR_LADDR_Msk) | 0x1F;
9          if (addr >= base && addr <= limit) return false;
10     }
11     return true; /* not in any NS/NSC region -> Secure */
12 }

```

Call from main_secure.c before releasing the NS core:

```

1  uart_printf("[S] 0x00010000 is %s\r\n",
2      TZM_IsSecure(0x00010000) ? "SECURE" : "NS/NSC");
3  uart_printf("[S] 0x00050000 is %s\r\n",
4      TZM_IsSecure(0x00050000) ? "SECURE" : "NS/NSC");
5  uart_printf("[S] 0x0003FE00 is %s (NSC)\r\n",
6      TZM_IsSecure(0x0003FE00) ? "SECURE" : "NS/NSC");

```

✓ Checkpoint A

UART output showing correct Secure/NS attribution for the three test addresses.

6. Part B — Implement and Call Secure API

6.1. Step 1 — Write the Secure API

In secure/src/secure_api.c:

```

1  #include <arm_cmse.h>
2  #include "secure_api.h"
3
4  /* Secure ADC value -- NS world must not read this directly */
5  static volatile int32_t s_last_adc_value = 0;
6
7  /* NSC entry function -- callable from Non-Secure world */
8  __attribute__((cmse_nonsecure_entry))
9  int32_t Secure_ADC_Read(void)
10 {
11     s_last_adc_value = (s_last_adc_value + 37) % 4096;
12     return s_last_adc_value;
13 }
14
15 __attribute__((cmse_nonsecure_entry))
16 void Secure_ADC_Init(void)
17 {
18     uart_printf("[S] Secure_ADC_Init called from NS world\r\n");
19     s_last_adc_value = 0;
20 }

```

In secure/src/secure_api.h:

```

1 #ifndef SECURE_API_H
2 #define SECURE_API_H
3 #include <stdint.h>
4
5 /* Declarations for NS world -- placed in CMSE import library */
6 int32_t Secure_ADC_Read(void);
7 void Secure_ADC_Init(void);
8
9 #endif /* SECURE_API_H */

```

6.2. Step 2 — Build the Secure Image and Generate the Import Library

```

cd secure
mingw32-make
# Output: secure.elf + cmse_implib.o (NSC veneer import library)

```

The linker flags `-Wl,-cmse-implib -Wl,-out-implib=cmse_implib.o` generate the import library containing SG stubs.

Verify SG stubs appear in the map file:

```
grep ".gnu.sgstubs" secure.map
```

Expected output:

.gnu.sgstubs	0x0003fe00	0x10	Secure_ADC_Read
.gnu.sgstubs	0x0003fe10	0x08	Secure_ADC_Init

✓ Checkpoint B1

Map file excerpt showing SG stubs located in the NSC region (0x0003FE00).

6.3. Step 3 — Call from Non-Secure FreeRTOS Task

Copy `cmse_implib.o` to `nonsecure/`. In `nonsecure/src/ns_tasks.c`:

```

1 #include "secure_api.h"
2
3 void vSensorTask(void *pv)
4 {
5     Secure_ADC_Init();
6     for (;;) {
7         int32_t value = Secure_ADC_Read();
8         uart_printf("[NS] ADC value = %ld\r\n", value);
9         vTaskDelay(pdMS_TO_TICKS(200));
10    }
11 }

```

Build the NS image (links against `cmse_implib.o`):

```

cd nonsecure
mingw32-make

```

6.4. Step 4 — Flash and Run

Flash the secure image first, then the non-secure image:

```

cd secure
mingw32-make flash

```

```
cd ../nonsecure
mingw32-make flash
```

Note

Flash order matters. The Secure image configures the SAU and releases the NS core. Always flash secure/ before nonsecure/ or the NS application will start without a valid SAU configuration.

Expected UART output:

```
[S] SAU configured
[S] 0x00010000 is SECURE
[S] 0x00050000 is NS/NSC
[S] Releasing Non-Secure image at 0x00040000
[NS] starting FreeRTOS...
[S] Secure_ADC_Init called from NS world
[NS] ADC value = 37
[NS] ADC value = 74
```

Checkpoint B2

UART output showing alternating [S] init and [NS] sensor values.

7. Part C — Trigger a SecureFault

7.1. Step 1 — Direct NS→Secure SRAM Access

In vSensorTask, add a deliberate access to Secure SRAM (this will fault):

```
1 /* Attempting to read Secure SRAM from NS world -- should
   SecureFault */
2 volatile uint32_t *secure_ptr = (volatile uint32_t *)0x20010000U;
3 uint32_t val = *secure_ptr; /* <- triggers SecureFault */
4 uart_printf("[NS] should never print: %lu\r\n", val);
```

7.2. Step 2 — Implement the SecureFault Handler

In nonsecure/src/main_ns.c:

```
1 void SecureFault_Handler(void)
2 {
3     uart_printf("[NS] !!! SecureFault !!!\r\n");
4     uart_printf("[NS] SFSR = 0x%08lX\r\n", SAU->SFSR);
5     uart_printf("[NS] SFAR = 0x%08lX\r\n", SAU->SFAR);
6     for (;;) {}
7 }
```

Checkpoint C

UART showing !!! SecureFault !!! with SFSR/SFAR values. Note the faulting address (0x20010000) reported in SFAR.

Question C1

The SecureFault is raised on the **NS** side (not the Secure side). Explain why — which hardware unit detects the violation and in which security state does the fault handler run?

? Question C2

The SFSR (Secure Fault Status Register) has several bits. Look up the meaning of bit 1 (SFARVALID) and bit 0 (INVEP). Which bit is set in your fault? What does it indicate?

8. Part D — Measure NSC Call Overhead**8.1. Step 1 — Benchmark NSC vs. Direct Call**

Use the DWT cycle counter (Lab 06) to compare NSC and direct call latency:

```

1 void vBenchmarkTask(void *pv)
2 {
3     DWT_Enable();
4     uint32_t nsc_cycles, direct_cycles;
5
6     /* NSC call overhead */
7     DWT_START();
8     volatile int32_t v1 = Secure_ADC_Read(); /* crosses S/NS
9         boundary */
10    DWT_STOP();
11    nsc_cycles = _dwt_elapsed();
12
13    /* Direct function call (NS-only function, same logic) */
14    DWT_START();
15    volatile int32_t v2 = ns_dummy_adc_read();
16    DWT_STOP();
17    direct_cycles = _dwt_elapsed();
18
19    uart_printf("[BENCH] NSC: %lu cycles   Direct: %lu cycles
20                Overhead: %lu\r\n",
21                nsc_cycles, direct_cycles, nsc_cycles -
22                direct_cycles);
23    vTaskDelete(NULL);
24 }

```

✓ Checkpoint D

Terminal output showing NSC and direct call cycle counts with computed overhead.

? Question D1

The NSC call overhead includes BLXNS + SG instruction + context save. At 150 MHz, convert the overhead cycles to nanoseconds. Is this acceptable for a 1 kHz control loop?

? Question D2

FreeRTOS-M33 (NS kernel) allocates a “Secure context” per NS task that calls NSC functions. Where is this context stored, and why is it needed?

9. Deliverables

#	Item	Details
1	Part A UART	SAU probe output — 3 addresses
2	Part B1 map excerpt	SG stubs in NSC region
3	Part B2 UART	Alternating [S] init and [NS] sensor values
4	Part C UART	SecureFault with SFSR/SFAR values
5	Part D UART	NSC vs. direct call cycle counts
6	Written answers	Questions C1–C2, D1–D2
7	Source listings	<code>secure_api.c</code> and <code>ns_tasks.c</code>

10. Key Concepts Demonstrated

Concept	Where you saw it
SAU region configuration	Part A
Security attribution verification	Part A <code>TZM_IsSecure</code>
CMSE <code>cmse_nonsecure_entry</code> + SG veneer	Part B
NSC import library and BLXNS call	Part B
SecureFault on illegal NS→S access	Part C
SFSR / SFAR fault diagnosis	Part C
NSC call overhead measurement	Part D

11. Implementation Notes

11.1. Board Hardware Init Declaration (`app.h`)

`BOARD_InitHardware()` is declared in `app.h` (from the SDK's `freertos_hello` example), *not* in `hardware_init.h`. Both the secure and non-secure main files must include it:

```
1 #include "board.h"
2 #include "app.h"           /* declares BOARD_InitHardware() */
```

Without this include the compiler emits: `error: implicit declaration of function 'BOARD_InitHardware'`

12. Reference

Resource	Relevant sections
ARM Cortex-M33 TRM	§B9 TrustZone-M, §B2.3 SecureFault
NXP AN13814	TrustZone-M on MCXN236
ARM CMSE specification	<code>cmse_nonsecure_entry</code> , BLXNS
GCC ARM options	<code>-mcmse</code> , <code>-cmse-implib</code> , <code>-out-implib</code>